

1. System architecture

Training fan-out — every client starts from the same immutable commit

Participant A	Hugging Face model repository	Participant B
Private dataset A	<code>main @ C_r</code>	Private dataset B
↓ exact-SHA download	model + config + round manifest	exact-SHA download ↓
download → own trainer → upload or trusted local plugin	versioned coordination plane	download → own trainer → upload or trusted local plugin
local checkpoint →	PR A PR B	← local checkpoint

Aggregation fan-in — only the owner publishes the global model

Inputs	Repository owner	Output
Pinned PR A commit Pinned PR B commit	Validate author allowlist, ancestry, round, schema, shapes and counts FedAvg + optional owner evaluation	Commit <code>C_(r+1)</code> to <code>main</code> Record manifest + optional tag

The Hub stores and transports checkpoints. It is not a privacy boundary: raw data stays local, but uploaded weights are visible to authorized readers.

2. Operation sequence

Step	Actor	Operation	Required guard
1	Owner	Run <code>init_repo.py</code> ; publish round 0	Record returned commit <code>c0</code>
2	Owner	Send repository ID and <code>c0</code> to A and B	Send the SHA—not “latest”
3	A + B	Run <code>client_download.py</code> , then any local trainer	Write a separate compatible checkpoint
4	A + B	Run <code>client_upload.py</code> to validate and open PRs	<code>parent_commit=c0</code> ; private data stays local
5	Owner	Discover open PRs or use explicit refs; pin SHAs	Allowlisted authors; current base + round
6	Owner	Dry-run validation, weighted FedAvg, optional evaluation	Reject stale, incompatible or non-finite updates
7	Owner	Publish aggregate and optional round tag	Commit to <code>main</code> with parent <code>c0</code>

Round transition

`main @ c0` → parallel client PRs → owner aggregation → `main @ c1` → repeat

If `main` changed after validation, Hugging Face rejects publication. The owner restarts from the new base instead of overwriting concurrent work.

3. Credential management

Actor	Capability	Credential policy
Owner: initialization	Create repository and first commit	Narrowly scoped owner write token
Participant	Read <code>c_r</code> ; authenticate and upload a PR	Each participant's own token; never the owner's token
Owner: aggregation	Read PR refs; update only <code>main</code>	Separate repository-scoped write token
Automated publisher	Publish one repository from CI	Prefer a short-lived trusted-publisher token

Resolution used by the scripts

`--token` → `HF_TOKEN` → active `hf auth login` cache

- Interactive workstation: use `hf auth login` , then verify with `hf auth whoami` .
- Automation: inject `HF_TOKEN` from a secret manager; mask logs and rotate it.
- Avoid `--token` for routine use because arguments may appear in history and process listings. Never commit tokens, `.env` files, or credential caches.
- Public repo: enable PRs. Private repo: grant explicit organization access.

4. Handling large models

The workflow supports one SafeTensors file or an indexed, deterministically sharded checkpoint. Aggregation processes one shard at a time.

Stage	Scalable design
Package	Deterministic sharded SafeTensors + index. All clients use the same architecture, names, dtypes and shard layout.
Transfer	Use <code>huggingface_hub >= 0.32</code> , exact revisions and a fast local Xet cache. Hub APIs handle large-file transfer.
Aggregate	Process one shard/tensor from every accepted client, accumulate in FP32/FP64 on CPU, write the output shard, then release input memory.
Finalize	Validate config, index, tensor schemas and non-finite values. Publish the model plus a round manifest with <code>parent_commit=C_r</code> .

Resource profile: memory is bounded by one output shard plus active tensors and the FP32/FP64 accumulator; disk holds every selected snapshot and one output.

Xet uses content-defined chunks around 64 KiB, deduplicates known chunks, and adapts transfer concurrency. Dense training may alter bytes throughout every weight file, so deduplication can be limited. Deltas or adapters reduce traffic only when the protocol defines how they are reconstructed and aggregated.